

Programmazione Sicura



Reverse
Engineering



Barbara Masucci

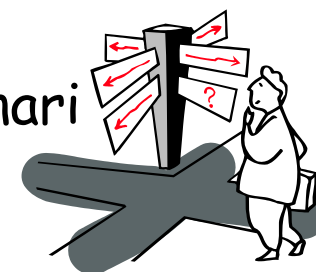
UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA

DIPARTIMENTO DI ECCELLENZA

Punto della situazione

- Nelle lezioni scorse abbiamo visto diverse tecniche per
 - Iniezione locale
 - Iniezione remota
 - Corruzione della memoria
- **Scopo della lezione di oggi:**
 - Scoprire come **estrarre informazioni** da file binari
 - Introdurre i principi di base della **Reverse Engineering**
 - Descrivere dei **tool** che possano essere utilizzati per la scoperta di vulnerabilità all'interno di programmi
 - Risolvere due **sfide** Capture The Flag presentate nell'ambito del programma **Cyberchallenge.IT**



Ottenere informazioni da un file binario

- Dato un **file binario**, possiamo
 - Controllare se è **eseguibile** oppure no
 - Scoprire per quale **architettura** è stato compilato
 - Collezionare **simboli e stringhe** usate dal programma
 - Controllare se c'è un **processo in esecuzione** associato al programma
 - Identificare i **nomi delle funzioni e delle librerie** usate dal programma



Ottenere informazioni da un file binario

- Esistono diversi comandi per estrarre informazioni da un **file binario**:
 - **file**
 - **strings**
 - **objdump**
 - **readelf**
- L'estrazione di tali informazioni non richiede l'esecuzione del file (**analisi statica**)



Ottenere informazioni da un file binario

- Il comando `file` determina il tipo del file binario e l'architettura per cui è stato compilato
- Uso: `file nomefile`

```
barbara@barbara-VirtualBox:~/Scaricati/SS2$ file /bin/bash
/bin/bash: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically link
ed, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]
=6f072e70e3e49380ff4d43cdde8178c24cf73daa, stripped
```



Ottenere informazioni da un file binario

- Il comando `strings` consente di collezionare tutte le stringhe presenti in un file binario
 - Stampa tutte le sequenze di caratteri stampabili con almeno 4 caratteri
- Le stringhe collezionate possono fornire informazioni su dati segreti
- Uso: `strings nomefile`



Ottenere informazioni da un file binario

- Ad esempio si consideri il seguente programma

```
CC> ./guessmyname
Guess my name: Paulo
I am sorry, this is not my name!
Please, try again.
CC> █
```

- Usando il comando **strings** si scopre facilmente il valore del nome da indovinare

```
CC> strings guessmyname
/lib64/ld-linux-x86-64.so.2
libc.so.6
__isoc99_scanf
puts
printf
malloc
__cxa_finalize
strcmp
__libc_start_main
GLIBC_2.7
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
u+UH
[1A\A1A^A_
Calef
Guess my name:
%10s
Well done! You have guessed my name!
I am sorry, this is not my name!
Please, try again.
:*3$"
GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.8060
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
```



Ottenere informazioni da un file binario

- Possiamo usare l'informazione ottenuta per "indovinare" il nome....

```
CC> ./guessmyname  
Guess my name: Calef  
Well done! You have guessed my name!  
CC> 
```



Ottenere informazioni da un file binario

- Non bisogna mai inserire **dati segreti** nel codice sorgente!

```
int main(int argc, char** argv) {  
    char* name = "Calef";  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s", input);  
    if (strcmp(name, input) == 0) {  
        printf("Well done! You have guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\n");  
        printf("Please, try again.\n");  
    }  
}
```



Ottenere informazioni da un file binario

- Il comando **objdump** fornisce diverse informazioni presenti in un file binario:
 - Informazioni presenti nell'header del file

```
CC> objdump -f /bin/bash

/bin/bash:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x00000000000030430
```

- Contenuto di specifiche sezioni del file

```
CC> objdump -s -j .rodata /bin/bash | more

/bin/bash:      file format elf64-x86-64

Contents of section .rodata:
de000 01000200 474e5520 62617368 2c207665  ....GNU bash, ve
de010 7273696f 6e202573 2d282573 290a0078  rsion %s-(%s)..x
de020 38365f36 342d7063 2d6c696e 75782d67  86_64-pc-linux-g
de030 6e750047 4e55206c 6f6e6720 6f707469  nu.GNU long opti
de040 6f6e733a 0a00092d 2d25730a 00536865  ons:....-%s..She
de050 6c6c206f 7074696f 6e733a0a 00092d25  ll options:...-%
de060 73206f72 202d6f20 6f707469 6f6e0a00  s or -o option..
de070 72756e5f 6f6e655f 636f6d6d 616e6400  run_one_command.
de080 2d630072 62617368 00492068 61766520  -c.rbash.I have
de090 6e6f206e 616d6521 003f3f68 6f73743f  no name!..??host?
1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-16-17-18-19-20-21-22-23-24-25-26-27-28-29-30-31-32-33-34-35-36-37-38-39-40-41-42-43-44-45-46-47-48-49-50-51-52-53-54-55-56-57-58-59-60-61-62-63-64-65-66-67-68-69-70-71-72-73-74-75-76-77-78-79-80-81-82-83-84-85-86-87-88-89-90-91-92-93-94-95-96-97-98-99-100-101-102-103-104-105-106-107-108-109-110-111-112-113-114-115-116-117-118-119-120-121-122-123-124-125-126-127-128-129-130-131-132-133-134-135-136-137-138-139-140-141-142-143-144-145-146-147-148-149-150-151-152-153-154-155-156-157-158-159-160-161-162-163-164-165-166-167-168-169-170-171-172-173-174-175-176-177-178-179-180-181-182-183-184-185-186-187-188-189-190-191-192-193-194-195-196-197-198-199-200-201-202-203-204-205-206-207-208-209-210-211-212-213-214-215-216-217-218-219-220-221-222-223-224-225-226-227-228-229-230-231-232-233-234-235-236-237-238-239-240-241-242-243-244-245-246-247-248-249-250-251-252-253-254-255-256-257-258-259-260-261-262-263-264-265-266-267-268-269-270-271-272-273-274-275-276-277-278-279-280-281-282-283-284-285-286-287-288-289-290-291-292-293-294-295-296-297-298-299-300-301-302-303-304-305-306-307-308-309-310-311-312-313-314-315-316-317-318-319-320-321-322-323-324-325-326-327-328-329-330-331-332-333-334-335-336-337-338-339-340-341-342-343-344-345-346-347-348-349-350-351-352-353-354-355-356-357-358-359-360-361-362-363-364-365-366-367-368-369-370-371-372-373-374-375-376-377-378-379-380-381-382-383-384-385-386-387-388-389-390-391-392-393-394-395-396-397-398-399-400-401-402-403-404-405-406-407-408-409-410-411-412-413-414-415-416-417-418-419-420-421-422-423-424-425-426-427-428-429-430-431-432-433-434-435-436-437-438-439-440-441-442-443-444-445-446-447-448-449-450-451-452-453-454-455-456-457-458-459-460-461-462-463-464-465-466-467-468-469-470-471-472-473-474-475-476-477-478-479-480-481-482-483-484-485-486-487-488-489-490-491-492-493-494-495-496-497-498-499-500-501-502-503-504-505-506-507-508-509-510-511-512-513-514-515-516-517-518-519-520-521-522-523-524-525-526-527-528-529-530-531-532-533-534-535-536-537-538-539-540-541-542-543-544-545-546-547-548-549-550-551-552-553-554-555-556-557-558-559-560-561-562-563-564-565-566-567-568-569-570-571-572-573-574-575-576-577-578-579-580-581-582-583-584-585-586-587-588-589-590-591-592-593-594-595-596-597-598-599-600-601-602-603-604-605-606-607-608-609-610-611-612-613-614-615-616-617-618-619-620-621-622-623-624-625-626-627-628-629-630-631-632-633-634-635-636-637-638-639-640-641-642-643-644-645-646-647-648-649-650-651-652-653-654-655-656-657-658-659-660-661-662-663-664-665-666-667-668-669-670-671-672-673-674-675-676-677-678-679-680-681-682-683-684-685-686-687-688-689-690-691-692-693-694-695-696-697-698-699-700-701-702-703-704-705-706-707-708-709-710-711-712-713-714-715-716-717-718-719-720-721-722-723-724-725-726-727-728-729-730-731-732-733-734-735-736-737-738-739-740-741-742-743-744-745-746-747-748-749-750-751-752-753-754-755-756-757-758-759-760-761-762-763-764-765-766-767-768-769-770-771-772-773-774-775-776-777-778-779-780-781-782-783-784-785-786-787-788-789-790-791-792-793-794-795-796-797-798-799-800-801-802-803-804-805-806-807-808-809-810-811-812-813-814-815-816-817-818-819-820-821-822-823-824-825-826-827-828-829-830-831-832-833-834-835-836-837-838-839-840-841-842-843-844-845-846-847-848-849-850-851-852-853-854-855-856-857-858-859-860-861-862-863-864-865-866-867-868-869-870-871-872-873-874-875-876-877-878-879-880-881-882-883-884-885-886-887-888-889-890-891-892-893-894-895-896-897-898-899-900-901-902-903-904-905-906-907-908-909-910-911-912-913-914-915-916-917-918-919-920-921-922-923-924-925-926-927-928-929-930-931-932-933-934-935-936-937-938-939-940-941-942-943-944-945-946-947-948-949-950-951-952-953-954-955-956-957-958-959-960-961-962-963-964-965-966-967-968-969-970-971-972-973-974-975-976-977-978-979-980-981-982-983-984-985-986-987-988-989-990-991-992-993-994-995-996-997-998-999-1000-1001-1002-1003-1004-1005-1006-1007-1008-1009-1010-1011-1012-1013-1014-1015-1016-1017-1018-1019-1020-1021-1022-1023-1024-1025-1026-1027-1028-1029-1030-1031-1032-1033-1034-1035-1036-1037-1038-1039-1040-1041-1042-1043-1044-1045-1046-1047-1048-1049-1050-1051-1052-1053-1054-1055-1056-1057-1058-1059-1060-1061-1062-1063-1064-1065-1066-1067-1068-1069-1070-1071-1072-1073-1074-1075-1076-1077-1078-1079-1080-1081-1082-1083-1084-1085-1086-1087-1088-1089-1090-1091-1092-1093-1094-1095-1096-1097-1098-1099-1100-1101-1102-1103-1104-1105-1106-1107-1108-1109-1110-1111-1112-1113-1114-1115-1116-1117-1118-1119-1120-1121-1122-1123-1124-1125-1126-1127-1128-1129-1130-1131-1132-1133-1134-1135-1136-1137-1138-1139-1140-1141-1142-1143-1144-1145-1146-1147-1148-1149-1150-1151-1152-1153-1154-1155-1156-1157-1158-1159-1160-1161-1162-1163-1164-1165-1166-1167-1168-1169-1170-1171-1172-1173-1174-1175-1176-1177-1178-1179-1180-1181-1182-1183-1184-1185-1186-1187-1188-1189-1190-1191-1192-1193-1194-1195-1196-1197-1198-1199-1200-1201-1202-1203-1204-1205-1206-1207-1208-1209-1210-1211-1212-1213-1214-1215-1216-1217-1218-1219-1220-1221-1222-1223-1224-1225-1226-1227-1228-1229-1230-1231-1232-1233-1234-1235-1236-1237-1238-1239-1240-1241-1242-1243-1244-1245-1246-1247-1248-1249-1250-1251-1252-1253-1254-1255-1256-1257-1258-1259-1260-1261-1262-1263-1264-1265-1266-1267-1268-1269-1270-1271-1272-1273-1274-1275-1276-1277-1278-1279-1280-1281-1282-1283-1284-1285-1286-1287-1288-1289-1290-1291-1292-1293-1294-1295-1296-1297-1298-1299-1300-1301-1302-1303-1304-1305-1306-1307-1308-1309-1310-1311-1312-1313-1314-1315-1316-1317-1318-1319-1320-1321-1322-1323-1324-1325-1326-1327-1328-1329-1330-1331-1332-1333-1334-1335-1336-1337-1338-1339-1340-1341-1342-1343-1344-1345-1346-1347-1348-1349-1350-1351-1352-1353-1354-1355-1356-1357-1358-1359-1360-1361-1362-1363-1364-1365-1366-1367-1368-1369-1370-1371-1372-1373-1374-1375-1376-1377-1378-1379-1380-1381-1382-1383-1384-1385-1386-1387-1388-1389-1390-1391-1392-1393-1394-1395-1396-1397-1398-1399-1400-1401-1402-1403-1404-1405-1406-1407-1408-1409-1410-1411-1412-1413-1414-1415-1416-1417-1418-1419-1420-1421-1422-1423-1424-1425-1426-1427-1428-1429-1430-1431-1432-1433-1434-1435-1436-1437-1438-1439-1440-1441-1442-1443-1444-1445-1446-1447-1448-1449-1450-1451-1452-1453-1454-1455-1456-1457-1458-1459-1460-1461-1462-1463-1464-1465-1466-1467-1468-1469-1470-1471-1472-1473-1474-1475-1476-1477-1478-1479-1480-1481-1482-1483-1484-1485-1486-1487-1488-1489-1490-1491-1492-1493-1494-1495-1496-1497-1498-1499-1500-1501-1502-1503-1504-1505-1506-1507-1508-1509-1510-1511-1512-1513-1514-1515-1516-1517-1518-1519-1520-1521-1522-1523-1524-1525-1526-1527-1528-1529-1530-1531-1532-1533-1534-1535-1536-1537-1538-1539-1540-1541-1542-1543-1544-1545-1546-1547-1548-1549-1550-1551-1552-1553-1554-1555-1556-1557-1558-1559-1560-1561-1562-1563-1564-1565-1566-1567-1568-1569-1570-1571-1572-1573-1574-1575-1576-1577-1578-1579-1580-1581-1582-1583-1584-1585-1586-1587-1588-1589-1590-1591-1592-1593-1594-1595-1596-1597-1598-1599-1600-1601-1602-1603-1604-1605-1606-1607-1608-1609-1610-1611-1612-1613-1614-1615-1616-1617-1618-1619-1620-1621-1622-1623-1624-1625-1626-1627-1628-1629-1630-1631-1632-1633-1634-1635-1636-1637-1638-1639-1640-1641-1642-1643-1644-1645-1646-1647-1648-1649-1650-1651-1652-1653-1654-1655-1656-1657-1658-1659-1660-1661-1662-1663-1664-1665-1666-1667-1668-1669-1670-1671-1672-1673-1674-1675-1676-1677-1678-1679-1680-1681-1682-1683-1684-1685-1686-1687-1688-1689-1690-1691-1692-1693-1694-1695-1696-1697-1698-1699-1700-1701-1702-1703-1704-1705-1706-1707-1708-1709-1710-1711-1712-1713-1714-1715-1716-1717-1718-1719-1720-1721-1722-1723-1724-1725-1726-1727-1728-1729-1730-1731-1732-1733-1734-1735-1736-1737-1738-1739-1740-1741-1742-1743-1744-1745-1746-1747-1748-1749-1750-1751-1752-1753-1754-1755-1756-1757-1758-1759-1760-1761-1762-1763-1764-1765-1766-1767-1768-1769-1770-1771-1772-1773-1774-1775-1776-1777-1778-1779-1780-1781-1782-1783-1784-1785-1786-1787-1788-1789-1790-1791-1792-1793-1794-1795-1796-1797-1798-1799-1800-1801-1802-1803-1804-1805-1806-1807-1808-1809-1810-1811-1812-1813-1814-1815-1816-1817-1818-1819-1820-1821-1822-1823-1824-1825-1826-1827-1828-1829-1830-1831-1832-1833-1834-1835-1836-1837-1838-1839-1840-1841-1842-1843-1844-1845-1846-1847-1848-1849-1850-1851-1852-1853-1854-1855-1856-1857-1858-1859-1860-1861-1862-1863-1864-1865-1866-1867-1868-1869-1870-1871-1872-1873-1874-1875-1876-1877-1878-1879-1880-1881-1882-1883-1884-1885-1886-1887-1888-1889-1890-1891-1892-1893-1894-1895-1896-1897-1898-1899-1900-1901-1902-1903-1904-1905-1906-1907-1908-1909-1910-1911-1912-1913-1914-1915-1916-1917-1918-1919-1920-1921-1922-1923-1924-1925-1926-1927-1928-1929-1930-1931-1932-1933-1934-1935-1936-1937-1938-1939-1940-1941-1942-1943-1944-1945-1946-1947-1948-1949-1950-1951-1952-1953-1954-1955-1956-1957-1958-1959-1960-1961-1962-1963-1964-1965-1966-1967-1968-1969-1970-1971-1972-1973-1974-1975-1976-1977-1978-1979-1980-1981-1982-1983-1984-1985-1986-1987-1988-1989-1990-1991-1992-1993-1994-1995-1996-1997-1998-1999-2000-2001-2002-2003-2004-2005-2006-2007-2008-2009-2010-2011-2012-2013-2014-2015-2016-2017-2018-2019-2020-2021-2022-2023-2024-2025-2026-2027-2028-2029-2030-2031-2032-2033-2034-2035-2036-2037-2038-2039-2040-2041-2042-2043-2044-2045-2046-2047-2048-2049-2050-2051-2052-2053-2054-2055-2056-2057-2058-2059-2060-2061-2062-2063-2064-2065-2066-2067-2068-2069-2070-2071-2072-2073-2074-2075-2076-2077-2078-2079-2080-2081-2082-2083-2084-2085-2086-2087-2088-2089-2090-2091-2092-2093-2094-2095-2096-2097-2098-2099-2100-2101-2102-2103-2104-2105-2106-2107-2108-2109-2110-2111-2112-2113-2114-2115-2116-2117-2118-2119-2120-2121-2122-2123-2124-2125-2126-2127-2128-2129-2130-2131-2132-2133-2134-2135-2136-2137-2138-2139-2140-2141-2142-2143-2144-2145-2146-2147-2148-2149-2150-2151-2152-2153-2154-2155-2156-2157-2158-2159-2160-2161-2162-2163-2164-2165-2166-2167-2168-2169-2170-2171-2172-2173-2174-2175-2176-2177-2178-2179-2180-2181-2182-2183-2184-2185-2186-2187-2188-2189-2190-2191-2192-2193-2194-2195-2196-2197-2198-2199-2200-2201-2202-2203-2204-2205-2206-2207-2208-2209-2210-2211-2212-2213-2214-2215-2216-2217-2218-2219-2220-2221-2222-2223-2224-2225-2226-2227-2228-2229-2230-2231-2232-2233-2234-2235-2236-2237-2238-2239-2240-2241-2242-2243-2244-2245-2246-2247-2248-2249-2250-2251-2252-2253-2254-2255-2256-2257-2258-2259-2260-2261-2262-2263-2264-2265-2266-2267-2268-2269-2270-2271-2272-2273-2274-2275-2276-2277-2278-2279-2280-2281-2282-2283-2284-2285-2286-2287-2288-2289-2290-2291-2292-2293-2294-2295-2296-2297-2298-2299-2300-2301-2302-2303-2304-2305-2306-2307-2308-2309-2310-2311-2312-2313-2314-2315-2316-2317-2318-2319-2320-2321-2322-2323-2324-2325-2326-2327-2328-2329-2330-2331-2332-2333-2334-2335-2336-2337-2338-2339-2340-2341-2342-2343-2344-2345-2346-2347-2348-2349-2350-2351-2352-2353-2354-2355-2356-2357-2358-2359-2360-2361-2362-2363-2364-2365-2366-2367-2368-2369-2370-2371-2372-2373-2374-2375-2376-2377-2378-2379-2380-2381-2382-2383-2384-2385-2386-2387-2388-2389-2390-2391-2392-2393-2394-2395-2396-2397-2398-2399-2400-2401-2402-2403-2404-2405-2406-2407-2408-2409-2410-2411-2412-2413-2414-2415-2416-2417-2418-2419-2420-2421-2422-2423-2424-2425-2426-2427-2428-2429-2430-2431-2432-2433-2434-2435-2436-2437-2438-2439-2440-2441-2442-2443-2444-2445-2446-2447-2448-2449-2450-2451-2452-2453-2454-2455-2456-2457-2458-2459-2460-2461-2462-2463-2464-2465-2466-2467-2468-2469-2470-2471-2472-2473-2474-2475-2476-2477-2478-2479-2480-2481-2482-2483-2484-2485-2486-2487-2488-2489-2490-2491-2492-2493-2494-2495-2496-2497-2498-2499-2500-2501-2502-2503-2504-2505-2506-2507-2508-2509-2510-2511-2512-2513-2514-2515-2516-2517-2518-2519-2520-2521-2522-2523-2524-2525-2526-2527-2528-2529-
```

Ottenere informazioni da un file binario

- Il comando **readelf** fornisce diverse informazioni presenti in un file binario:
 - Informazioni presenti nell'header del file

```
CC> readelf -h /bin/bash
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                          Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:                0x30430
  Start of program headers:          64 (bytes into file)
  Start of section headers:         1181528 (bytes into file)
  Flags:                              0x0
  Size of this header:                64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:          13
  Size of section headers:           64 (bytes)
  Number of section headers:          30
  Section header string table index: 29
```



Ottenere informazioni da un file binario

- Il comando `readelf` fornisce diverse informazioni presenti in un file binario:
 - Informazioni presenti in particolari sezioni del file

```
CC> readelf -x .rodata guessmyname

Hex dump of section '.rodata':
0x00002000 01000200 00000000 43616c65 66004775 .....Calef.Gu
0x00002010 65737320 6d79206e 616d653a 20002531 ess my name: .%1
0x00002020 30730000 00000000 57656c6c 20646f6e 0s.....Well don
0x00002030 65212059 6f752068 61766520 67756573 e! You have gues
0x00002040 73656420 6d79206e 616d6521 00000000 sed my name!....
0x00002050 4920616d 20736f72 72792c20 74686973 I am sorry, this
0x00002060 20697320 6e6f7420 6d79206e 616d6521 is not my name!
0x00002070 00506c65 6173652c 20747279 20616761 .Please, try aga
0x00002080 696e2e00                               in..

CC> 
```



Analisi statica e dinamica

- A volte però le informazioni ottenute con l'analisi statica non bastano per scoprire il **punto debole** di un programma
- In tali casi, è necessario esaminare il programma mentre è in esecuzione (**analisi dinamica**)
 - Un tool che consente tale esame è **gdb**



Reverse engineering

- Per analizzare a fondo il comportamento di un file eseguibile è utile dare uno sguardo al suo
 - codice assembly
 - codice sorgente
- Ciò può essere fatto mediante **disassemblatori** e **decompilatori**



Disassemblatori

I **disassemblatori** sono spesso utilizzati per analizzare programmi eseguibili alla ricerca di vulnerabilità



➤ Esempi: **gdb**, **objdump**

- Abbiamo visto un esempio di utilizzo nelle sfide CTF su stack-based buffer overflow
 - Abbiamo analizzato la memoria allocata al programma
 - Abbiamo investigato sulle relazioni spaziali delle variabili in uso
 - Abbiamo determinato il layout dello stack



Disassembatori

(gdb) disassemble main

Dump of assembler code for function main:

```
0x08048408 <main+0>: push    %ebp
0x08048409 <main+1>: mov     %esp,%ebp
0x0804840b <main+3>: and     $0xfffffffff0,%esp
0x0804840e <main+6>: sub     $0x50,%esp
0x08048411 <main+9>: lea     0x10(%esp),%eax%
0x08048415 <main+13>: mov     eax, (%esp)
0x08048418 <main+16>: call    0x804830c <gets@plt>
0x0804841d <main+21>: leave
0x0804841e <main+22>: ret
End of assembler dump.
```



Decompilatori

- I **decompilatori** rendono ancora più semplice la ricerca di vulnerabilità, poichè consentono l'ispezione del codice sorgente
- Il codice ottenuto talvolta necessita di essere reso più leggibile mediante
 - Modifica della **signature delle funzioni** (valore di ritorno e argomenti)
 - Modifica di nomi e tipi delle **variabili**



Ghidra

- Tra i tool disponibili per la decompilazione, useremo **Ghidra**
 - Si tratta di un tool rilasciato nel 2019 dall'NSA
 - E' un prodotto open-source, disponibile al link <https://ghidra-sre.org/>
 - Useremo la versione 9.2.2
 - Per usarlo è necessario innanzitutto installare **Java Development Kit 11**
 - **NOTA:** JDK 20 crea problemi!



Ghidra

- Ghidra non necessita di alcuna installazione
 - E' sufficiente scaricare il pacchetto ed estrarlo



- Dopo l'estrazione, basta avviare il programma
 - **ghidraRun.bat** su Windows
 - **ghidraRun** su Mac o Linux

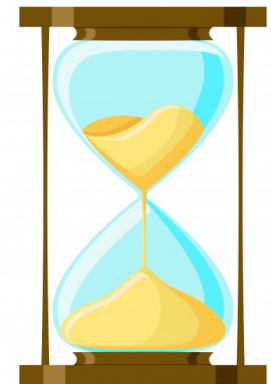
Viene caricata l'interfaccia grafica di Ghidra



CTF: Slow Printer

➤ Descrizione della sfida:

- Ci viene fornito un file binario che, mandato in esecuzione, procede alla stampa della bandierina da catturare
- Problema: ci vuole tempo, **molto tempo**...
- Riusciamo a catturare la bandierina in tempi brevi?



CTF: Slow Printer

➤ Innanzitutto, scarichiamo il file binario `slow_printer` relativo alla sfida sulla nostra macchina Linux

➤ Poi, visualizziamo i suoi metadati con `ls -la`

```
-rw-rw-r-- 1 barbara barbara 14560 mar 16 12:07 slow_printer
```

➤ Modifichiamo i permessi in modo che il file possa essere eseguito

`chmod u+x slow_printer`

```
-rwxrw-r-- 1 barbara barbara 14560 mar 16 12:07 slow_printer
```



CTF: Slow Printer

- Ora possiamo mandare il file in esecuzione

```
barbara@barbara-VirtualBox:~/Scaricati$ ./slow_printer
Your flag is:
CCIT{tim3}
```

- Il programma inizia a stampare la flag, ma la stampa sembra **rallentata**
- Probabilmente, c'è la chiamata a qualche funzione che provoca un **ritardo**



CTF: Slow Printer

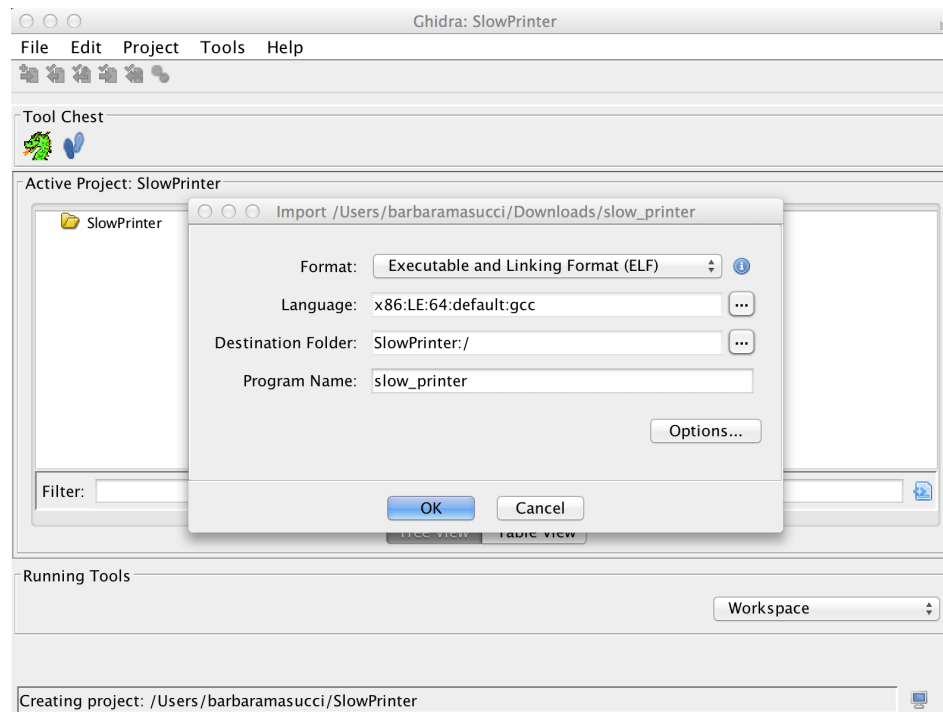
- Idea: "eliminare" la chiamata che ritarda l'esecuzione della stampa della flag
- Tra i tool disponibili, decidiamo di usare Ghidra



CTF: Slow Printer

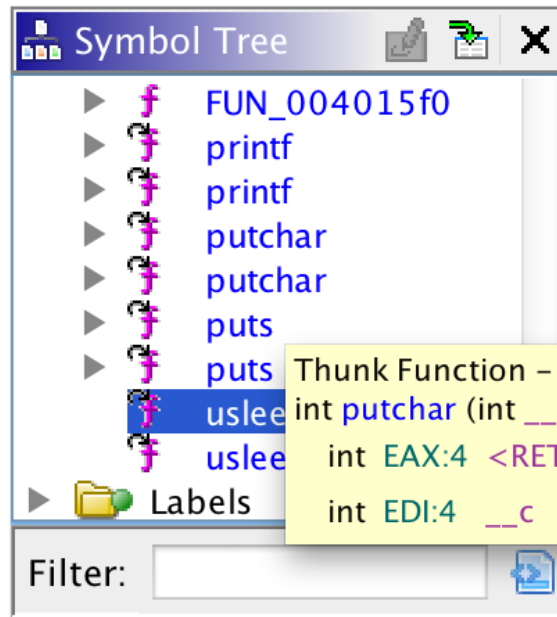
➤ Dopo aver lanciato Ghidra:

- New Project -> Non Shared e diamo un nome al progetto
- Trasciniamo il file `slow_printer` nel progetto



CTF: Slow Printer

- Da **Symbol Tree**, scegliamo **Functions** e vediamo la lista delle funzioni
 - Notiamo la funzione **usleep**



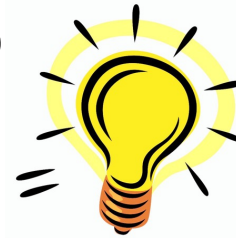
Decompile: usleep - (slow_p...

```
1
2 /* WARNING: Unknown calling convention ye
3
4 int usleep(__useconds_t __useconds)
5
6 {
7     int iVar1;
8
9     iVar1 = usleep(__useconds);
10    return iVar1;
11 }
12
```



CTF: Slow Printer

- Idea: Modificare il comportamento della funzione `usleep`



- Come fare?
 - Possiamo usare la variabile `LD_PRELOAD` per ridefinire dinamicamente la funzione `usleep` (function overriding) senza dover ricompilare il sorgente !



CTF: Slow Printer

- Il file `usleep.c` contiene una nuova implementazione della funzione `usleep`

```
#include <unistd.h>
#include <sys/types.h>

int usleep(useconds_t usec) {

    ;

}
```

- In pratica, la funzione non fa nulla



CTF: Slow Printer

- Creiamo il file `usleep.c` e scriviamo la nuova implementazione
`nano usleep.c`
- Ora generiamo la libreria condivisa usando `gcc`
`gcc -shared -fPIC -o usleep.so usleep.c`
- Opzioni usate
 - `shared`: genera un oggetto linkabile a tempo di esecuzione e condivisibile con altri oggetti
 - `fPIC`: genera codice indipendente dalla posizione (Position Independent Code), rilocabile ad un indirizzo di memoria arbitrario



CTF: Slow Printer

- Per caricare anticipatamente la **libreria condivisa** `usleep.so`, modifichiamo la variabile `LD_PRELOAD`:
`export LD_PRELOAD=./usleep.so`
- Ora mandiamo in esecuzione il programma
`./slow_printer`
- Il programma stampa la flag: sfida vinta!



CTF: Slow Printer

➤ Conclusioni:

- L'uso di un **tool di reversing** ci ha consentito di risalire al sorgente, a partire dal binario eseguibile
- Mediante ispezione del sorgente abbiamo compreso che la causa del ritardo della stampa era la funzione **usleep**
- Abbiamo quindi modificato il comportamento della funzione **usleep** vincendo la sfida



CTF: Pacman

- Descrizione della sfida:
 - Giochiamo a Pacman!
- Materiale fornito:
 - Binario eseguibile: **pacman**



CTF: Pacman

- Innanzitutto, scarichiamo il file binario **pacman** relativo alla sfida sulla nostra macchina Linux
- Poi, visualizziamo i suoi metadati con **ls -la**

```
-rw-rw-r-- 1 barbara barbara 14504 mar 23 10:57 pacman
```

- Modifichiamo i permessi in modo che il file possa essere eseguito

```
chmod u+x pacman
```

```
-rwxrw-r-- 1 barbara barbara 14504 mar 23 10:57 pacman
```



CTF: Pacman

- Mandiamo il file in esecuzione

```
barbara@barbara-VirtualBox:~/Scaricati$ ./pacman  
barbara@barbara-VirtualBox:~/Scaricati$
```

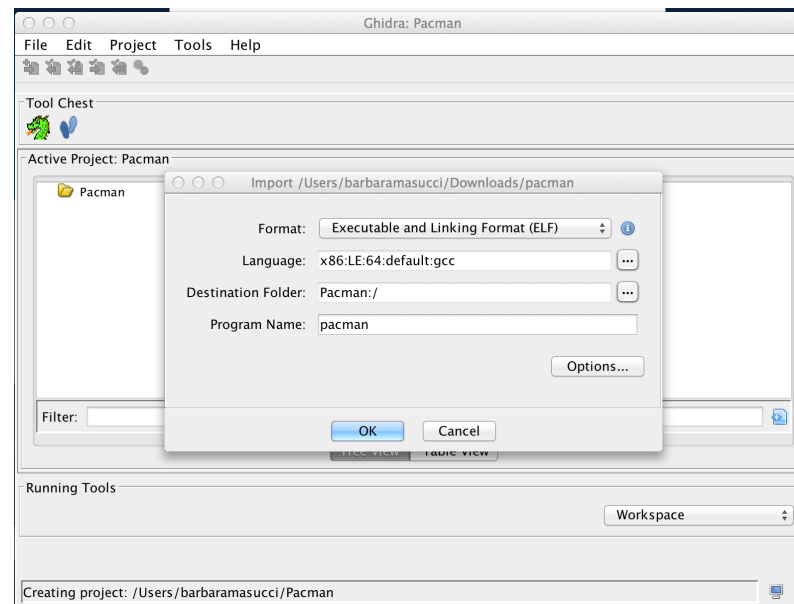
- Il programma termina anche se proviamo a passargli un input

```
barbara@barbara-VirtualBox:~/Scaricati$ ./pacman input  
Game over.  
barbara@barbara-VirtualBox:~/Scaricati$
```



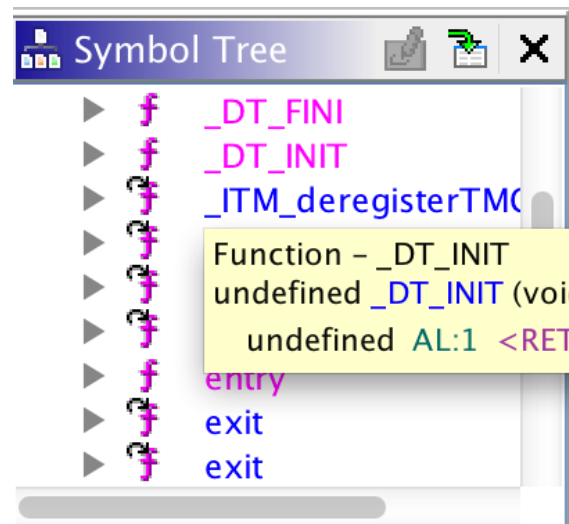
CTF: Pacman

- Analizzeremo il binario usando Ghidra
 - New Project -> Non Shared e diamo un nome al progetto
 - Trasciniamo il file **pacman** nel progetto



CTF: Pacman

- Da **Symbol Tree**, scegliamo **Functions** e vediamo la lista delle funzioni
- Poichè non è presente la funzione **main**, cerchiamo **entry**



CTF: Pacman

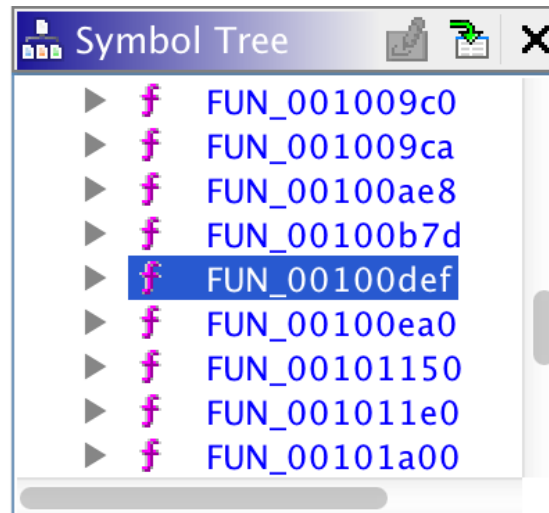
Decompiliamo **entry**

```
Decompile: entry - (pacman)
1
2 void entry(undefined8 param_1,undefined8 param_2,undefined8 param_3)
3
4 {
5     undefined8 in_stack_00000000;
6     undefined auStack8 [8];
7
8     __libc_start_main(FUN_00100def,in_stack_00000000,&stack0x00000008,FUN_00101a00,FUN_00101a70,
9                     param_3,auStack8);
10    do {
11        /* WARNING: Do nothing block with infinite loop */
12    } while( true );
13 }
14
```



CTF: Pacman

- Guardiamo quale funzione è invocata da `_libc_start_main()`
- Troviamo la funzione `FUN_00100def`
 - Cerchiamola nel Symbol Tree e decompiliamola



CTF: Pacman

Il codice non è molto leggibile

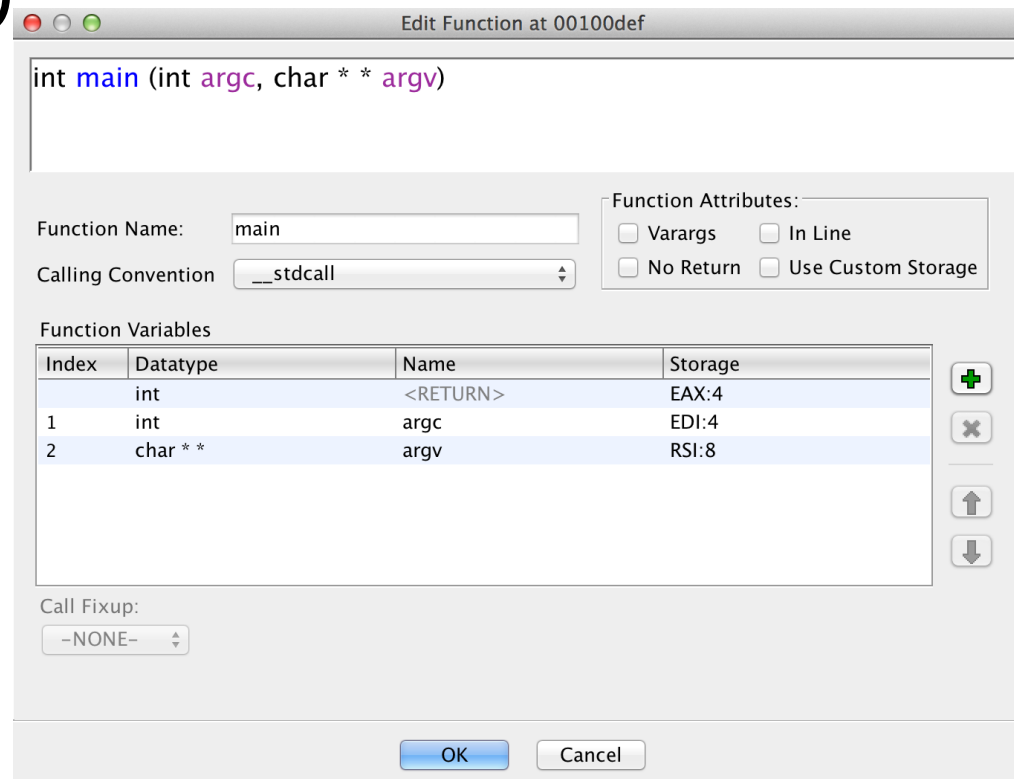
```
Decompile: FUN_00100def - (pacman)

1
2 undefined8 FUN_00100def(int param_1, long param_2)
3
4 {
5     long lVar1;
6     char *pcVar2;
7     undefined **local_10;
8
9     if ((param_1 == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
10         clock_gettime(0, (timespec *)&DAT_003030e0);
11         local_10 = &PTR_s_pacmanpacman_00303060;
12         while (*local_10 != (char *)0x0) {
13             pcVar2 = strdup(*local_10);
14             *local_10 = pcVar2;
15             local_10 = local_10 + 1;
16         }
17         FUN_00100b7d(*(undefined8 *) (param_2 + 8));
18         return 0;
19     }
20     return 0xffffffff;
21 }
22
```



CTF: Pacman

➤ Rendiamo il codice più leggibile, **modificando la signature della funzione** (facendo right-click su essa)



CTF: Pacman

Ora il codice del **main** appare così

```
Decompile: main - (pacman)

1
2 int main(int argc, char **argv)
3
4 {
5     long lVar1;
6     char *pcVar2;
7     undefined **local_10;
8
9     if ((argc == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
10         clock_gettime(0, (timespec *)&DAT_003030e0);
11         local_10 = &PTR_s_pacmanpacman_00303060;
12         while (*local_10 != (char *)0x0) {
13             pcVar2 = strdup(*local_10);
14             *local_10 = pcVar2;
15             local_10 = local_10 + 1;
16         }
17         FUN_00100b7d(argv[1]);
18         return 0;
19     }
20     return -1;
21 }
22
```



CTF: Pacman

- All'inizio del `main` notiamo un controllo sul numero di parametri in input al programma

```
C: Decompile: main - (pacman)
1
2 int main(int argc, char **argv)
3
4 {
5     long lVar1;
6     char *pcVar2;
7     undefined **local_10;
8
9     if ((argc == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
10         clock_gettime(0, (timespec *)&DAT_003030e0);
11         local_10 = &PTR_s_pacmanpacman_00303060;
12         while (*local_10 != (char *)0x0) {
13             pcVar2 = strdup(*local_10);
14             *local_10 = pcVar2;
15             local_10 = local_10 + 1;
16         }
17         FUN_00100b7d(argv[1]);
18         return 0;
19     }
20     return -1;
21 }
```

- Se il numero di parametri è diverso da 2, il programma termina

- Quindi il programma **si aspetta un input**



CTF: Pacman

- Poi, notiamo l'invocazione della chiamata di sistema **ptrace**

```
Decompile: main - (pacman)
1
2 int main(int argc, char **argv)
3
4 {
5     long lVar1;
6     char *pcVar2;
7     undefined **local_10;
8
9     if ((argc == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
10         clock_gettime(0, (timespec *)&DAT_003030e0);
11         local_10 = &PTR_s_pacmanpacman_00303060;
12         while (*local_10 != (char *)0x0) {
13             pcVar2 = strdup(*local_10);
14             *local_10 = pcVar2;
15             local_10 = local_10 + 1;
16         }
```

- Cosa fa la chiamata **ptrace**?

■ Leggiamo il manuale



CTF: Pacman

- La chiamata di sistema **ptrace** consente a un processo (**tracer**) di osservare l'esecuzione di un altro processo (**tracee**)

```
lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0),
```

- Nel nostro caso, il processo cerca di fare il **ptrace** di sè stesso
 - Quindi, indirettamente, controlla se c'è un altro processo (ad es. un **debugger**) attaccato ad esso
 - Se questo è il caso, **ptrace** dà in output -1
 - Si tratta di un **anti-debugging trick**



CTF: Pacman

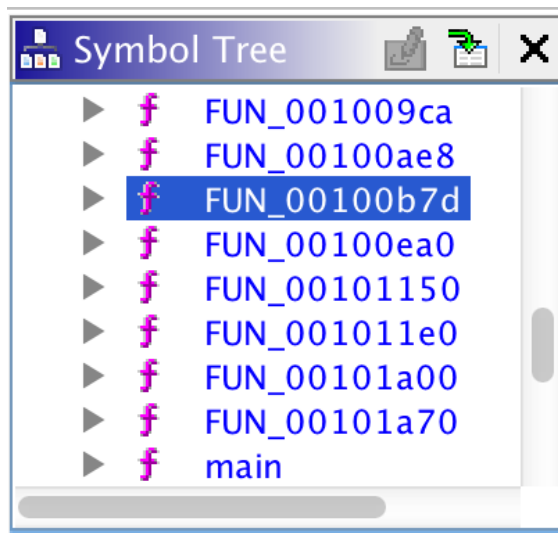
- Se il programma ha ricevuto un input e non c'è un debugger collegato al processo, si procede
 - Tralasciamo le altre istruzioni e passiamo all'invocazione della funzione `FUN_00100b7d`

```
Decompile: main - (pacman)
1
2 int main(int argc, char **argv)
3
4 {
5     long lVar1;
6     char *pcVar2;
7     undefined **local_10;
8
9     if ((argc == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
10         clock_gettime(0, (timespec *)&DAT_003030e0);
11         local_10 = &PTR_s_pacmanpacman_00303060;
12         while (*local_10 != (char *)0x0) {
13             pcVar2 = strdup(*local_10);
14             *local_10 = pcVar2;
15             local_10 = local_10 + 1;
16         }
17         FUN_00100b7d(argv[1]);
18         return 0;
19     }
20     return -1;
21 }
```



CTF: Pacman

- Troviamo la funzione **FUN_00100b7d**
- Cerchiamola nel Symbol Tree e decompiliamola

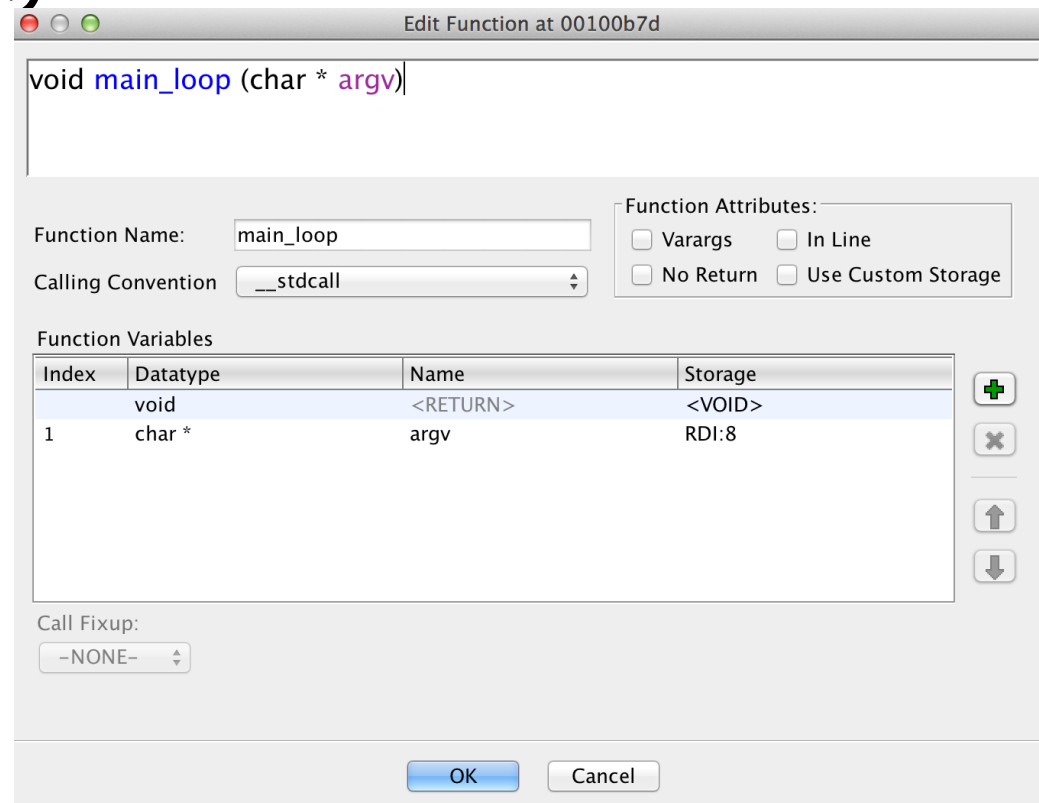


```
Decompile: FUN_00100b7d - (pacman)
1
2 void FUN_00100b7d(char *param_1)
3
4 {
5     char *pcVar1;
6     char cVar2;
7     char cVar3;
8     ushort **ppuVar4;
9     long in_FS_OFFSET;
10    char *local_150;
11    int local_144;
12    int local_140;
13    int local_13c;
14    timespec local_138;
15    char local_128 [4];
16    undefined local_124;
17    undefined local_123;
18    undefined local_122;
19    long local_20;
20
21    local_20 = *(long *) (in_FS_OFFSET + 0x28);
22    memset(local_128,0,0x100);
23    local_144 = 0;
24    local_140 = 1;
25    local_13c = 1;
26    local_150 = param_1;
27    do {
28        clock_gettime(0,&local_138);
29        cVar2 = (&PTR_s_pacmanpacman_00303060)[local_13c][local_140];
30        cVar3 = FUN_00100ae8(start_time,DAT_003030e8,local_138.tv_sec,local_138.tv_nsec);
31        cVar3 = cVar3 + cVar2;
32        switch(cVar3) {
33            case 'a':
34            case 'c':
35            case 'm':
36            case 'n':
37            case 'p':
```



CTF: Pacman

➤ Rendiamo il codice più leggibile, **modificando la signature della funzione** (facendo right-click su essa)



CTF: Pacman



Notiamo il blocco di codice finale

- I caratteri 'j', 'k', 'h', 'l' incrementano e decrementano due variabili (`local_13c` e `local_140`)
- Questi caratteri sono usati nell'editor `vi` per muovere il cursore

```
63     if (cVar2 == 'j') {
64         local_13c = local_13c + 1;
65     }
66     else {
67         if (cVar2 < 'k') {
68             if (cVar2 == 'h') {
69                 local_140 = local_140 + -1;
70             }
71         }
72         else {
73             if (cVar2 == 'k') {
74                 local_13c = local_13c + -1;
75             }
76             else {
77                 if (cVar2 == 'l') {
78                     local_140 = local_140 + 1;
79                 }
80             }
81         }
82     }
83 } while( true );
84 }
85
```



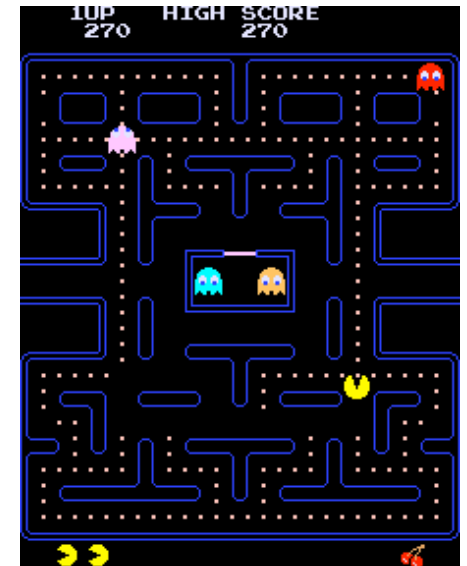
CTF: Pacman

➤ Poichè il nome della sfida è **Pacman**...
può darsi che questi caratteri servano
a **muovere il cursore** in una sorta di labirinto?



CTF: Pacman

- Prima abbiamo visto che **il programma si aspetta un input**
- Deduciamo che l'input debba essere una sequenza di caratteri **'j', 'k', 'h', 'l'**
- Quale sequenza ci fa vincere la sfida?



CTF: Pacman

➤ Tornando in alto nel codice, rinominiamo le variabili

- `local_13c` → `y`
- `local_140` → `x`
- `PTR_s_pacmanpacman_003030360` → `maze`

```
local_144 = 0;  
local_140 = 1;  
local_13c = 1;  
local_150 = param_1;  
do {  
    clock_gettime(0, &local_138);  
    cVar2 = (&PTR_s_pacmanpacman_00303060)[local_13c][local_140];
```



CTF: Pacman

➤ Il codice ora appare così

```
x = 1;
y = 1;
local_150 = param_1;
do {
    clock_gettime(0,&local_138);
    cVar2 = (&maze)[y][x];
```

- Quindi, il gioco parte dalla casella (1,1) e la posizione cambia in base all'input dato al programma (`argv[1]`)
- Più in basso, notiamo un controllo sul valore della casella su cui si finisce in base all'input fornito



CTF: Pacman

Se si finisce su una delle caselle contenenti uno dei caratteri della parola "pacman", il programma termina e stampa

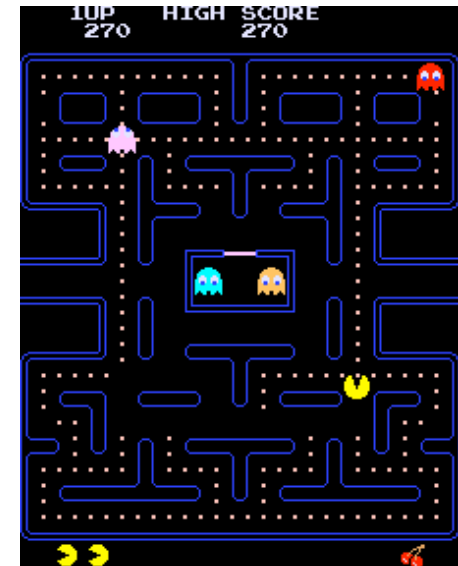
Game over

```
switch(cVar3) {  
case 'a':  
case 'c':  
case 'm':  
case 'n':  
case 'p':  
switchD_00100c74_caseD_61:  
    puts("Game over.");  
    /* WARNING: Subroutine does not return */  
    exit(-1);
```



CTF: Pacman

- Quindi dobbiamo spostarci nel labirinto senza toccare le caselle contenenti i caratteri 'a', 'c', 'm', 'n', 'p'
- Per capire come muoverci, dobbiamo esaminare il contenuto della variabile `maze`



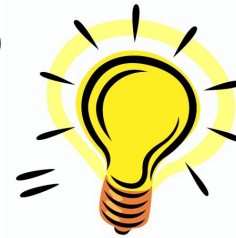
CTF: Pacman

- Per esaminare il contenuto di `maze` c'è necessità che il programma sia in esecuzione
- Potremmo usare `gdb` ma...come aggirare l'anti-debugging trick?
- Se non troviamo una soluzione, `ptrace` individuerà la presenza del debugger e il programma terminerà



CTF: Pacman

- Idea: Modificare il comportamento della funzione `ptrace`



- Come fare?
 - Possiamo usare la variabile `LD_PRELOAD` per ridefinire dinamicamente la funzione `ptrace` (function overriding) senza dover ricompilare il sorgente



CTF: Pacman

- Il file `ptrace.c` contiene una nuova implementazione delle funzione `ptrace`

```
#include <unistd.h>
#include <sys/types.h>

int ptrace(int i, int j, int k, int l) {

    return 0;

}
```

- In pratica, la funzione non fa nulla



CTF: Pacman

- Creiamo il file `ptrace.c` e scriviamo la nuova implementazione
`nano ptrace.c`
- Ora generiamo la libreria condivisa usando `gcc`
`gcc -shared -fPIC -o ptrace.so ptrace.c`
- Opzioni usate
 - `shared`: genera un oggetto linkabile a tempo di esecuzione e condivisibile con altri oggetti
 - `fPIC`: genera codice indipendente dalla posizione (Position Independent Code), rilocabile ad un indirizzo di memoria arbitrario



CTF: Pacman

- Prima di avviare **gdb**, assicuriamoci di aver installato **GEF** (**G**DB **E**nhanced **F**eature)
 - <https://gef.readthedocs.io/en/master/>
- **GEF** arricchisce i comandi di **gdb** e semplifica i processi di
 - Analisi dinamica
 - Sviluppo di exploit



CTF: Pacman

- Avviamo **gdb** per analizzare **pacman**
gdb -q pacman

```
barbara@barbara-VirtualBox: ~/Scaricati
barbara@barbara-VirtualBox:~/Scaricati$ gdb -q pacman
GEF for linux ready, type `gef' to start, `gef config' to configure
87 commands loaded for GDB 7.11.1 using Python engine 3.5
[*] 5 commands could not be loaded, run `gef missing' to know why.
GEF for linux ready, type `gef' to start, `gef config' to configure
87 commands loaded for GDB 7.11.1 using Python engine 3.5
[*] 5 commands could not be loaded, run `gef missing' to know why.
Reading symbols from pacman...(no debugging symbols found)...done.
gef> 
```

- Il programma si arresta in attesa di un comando



CTF: Pacman

- Inseriamo un breakpoint alla funzione invocata subito dopo `ptrace` nel `main`

```
int main(int argc, char **argv)
{
    long lVar1;
    char *pcVar2;
    undefined **local_10;

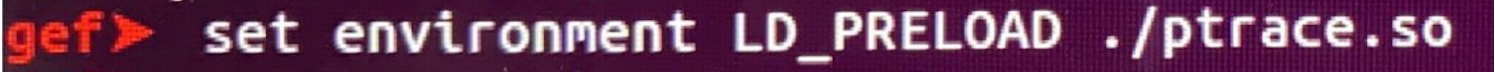
    if ((argc == 2) && (lVar1 = ptrace(PTRACE_TRACEME, 0, 1, 0), lVar1 != -1)) {
        clock_gettime(0, (timespec *)&DAT_003030e0);
    }
}
```

```
gef> b clock_gettime
Breakpoint 1 at 0x840
gef> 
```

- Quando `pacman` sarà in esecuzione, si arresterà al breakpoint



CTF: Pacman

- Prima di mandare `pacman` in esecuzione, dobbiamo aggirare il controllo anti-debugging di `ptrace`
- Settiamo la variabile di ambiente `LD_PRELOAD`
`set environment LD_PRELOAD ./ptrace.so`

- In tal modo provocheremo l'esecuzione della `ptrace` modificata (che non fa nulla)



CTF: Pacman

- Ora mandiamo `pacman` in esecuzione con un input qualsiasi

`run input`

```
[#0] Id 1, Name: "pacman", stopped 0x7ffff7920910 in __GI__clock_gettime (), reason: BREAKPOINT
[trace]
[#0] 0x7ffff7920910 → __GI__clock_gettime(clock_id=0x0, tp=0x5555557570e0)
[#1] 0x555555554e40 → lea rax, [rip+0x202219] # 0x555555757060
[#2] 0x7ffff782b840 → __libc_start_main(main=0x555555554def, argc=0x2, argv=0x7fffffffde98, init=<optimized out>, fini=<optimized out>, rtld_fini=<optimized out>, stack_end=0x7fffffffde88)
[#3] 0x5555555548ea → hlt
gef> |
```

- Il programma, dopo l'esecuzione della `ptrace` modificata, si ferma sul breakpoint (`clock_gettime`)

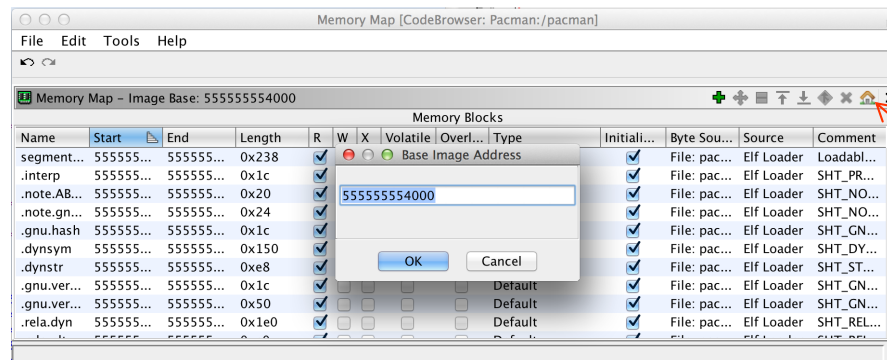


CTF: Pacman

- Recuperiamo l'indirizzo di partenza del processo in memoria con **vmm**

```
gef> vmm
[ Legend: Code | Heap | Stack ]
Start      End      Offset
0x0000555555554000 0x0000555555556000 0x0000000000000000
```

- Sincronizziamo **Ghidra** con **gdb**
 - Selezioniamo Window → Memory
 - Clicchiamo sulla casetta e inseriamo l'indirizzo



CTF: Pacman

- In **Ghidra** vediamo qual è l'indirizzo dell'istruzione successiva all'accesso alla variabile **maze**

55555554bfb	48 8d 05	LEA	RAX, [maze]	29	cVar2 = (&maze)[y][x];
	5f 24 20 00			30	auVar4 = FUN_55555554ae8
55555554c01	48 8b 14 02	MOV	RDX, qword ptr [RDX + RAX*0x1] => PTR_DAT_55555554c01	31	cVar2 = SUB161(auVar4, 0)



- In **gdb**, inseriamo un breakpoint a tale indirizzo

b *0x0000555555554c01

```
gef> b *0x0000555555554c01
Breakpoint 2 at 0x555555554c01
gef> |
```



CTF: Pacman

- Proseguiamo con l'esecuzione del programma finchè non giungiamo **al breakpoint 2**
 - In tal modo ci troveremo al punto in cui il programma accede alla variabile **maze**
- E' necessario digitare il comando **c** per 4 volte
 - Il programma si ferma tre volte sul **breakpoint 1** e la quarta volta sul **breakpoint 2**



CTF: Pacman

- Ora visualizziamo il contenuto del registro `rax`
`telescope $rax`

```
gef> telescope $rax
0x0000555555757060 +0x0000: 0x0000555555758010 → "pacmanpacman"
0x0000555555757068 +0x0008: 0x0000555555758030 → "cho4$aaagioc"
0x0000555555757070 +0x0010: 0x0000555555758050 → "caaarmmmmmmc"
0x0000555555757078 +0x0018: 0x0000555555758070 → "cz1pia66600c"
0x0000555555757080 +0x0020: 0x0000555555758090 → "cx4p2c00666c"
0x0000555555757088 +0x0028: 0x00005555557580b0 → "cg8a2pacmanc"
0x0000555555757090 +0x0030: 0x00005555557580d0 → "ci_cz737373c"
0x0000555555757098 +0x0038: 0x00005555557580f0 → "co1pacpaconc"
0x00005555557570a0 +0x0040: 0x0000555555758110 → "c4____pac0ac"
0x00005555557570a8 +0x0048: 0x0000555555758130 → "czxgioN1234c"
gef> 
```

- Abbiamo ottenuto il contenuto della variabile
`maze`



CTF: Pacman

Casella (1,1)
di maze
(labirinto)

Guardando il contenuto di `maze`, capiamo che la sequenza giusta da fornire in input è

`111jjjjj11111jjjhhh`

```
gef> telescope $rax
0x0000555555757060 +0x0000: 0x0000555555758010 → "pacmanpacman"
0x0000555555757068 +0x0008: 0x0000555555758030 → "che4$aaagioc"
0x0000555555757070 +0x0010: 0x0000555555758050 → "caaarmmmmmc"
0x0000555555757078 +0x0018: 0x0000555555758070 → "cz1pia66600c"
0x0000555555757080 +0x0020: 0x0000555555758090 → "cx4p2c00666c"
0x0000555555757088 +0x0028: 0x00005555557580b0 → "cg8a2pacmanc"
0x0000555555757090 +0x0030: 0x00005555557580d0 → "ci_cz737373c"
0x0000555555757098 +0x0038: 0x00005555557580f0 → "co1pacpaconc"
0x00005555557570a0 +0x0040: 0x0000555555758110 → "c4____pac0ac"
0x00005555557570a8 +0x0048: 0x0000555555758130 → "czxgion1234c"
gef> 
```



CTF: Pacman

- Usciamo da `gdb` (comando `q`) e digitiamo
`./pacman 111jjjjj11111jjjhhh`
- Il programma stampa la flag: Sfida vinta!



CTF: Pacman

➤ Conclusioni:

- L'uso di un **tool di reversing** ci ha consentito di risalire al sorgente, a partire dall'eseguibile
- Mediante ispezione del sorgente abbiamo notato la presenza di un anti-debugging trick (funzione **ptrace**)
- Abbiamo quindi modificato il comportamento della funzione **ptrace** per poter esaminare il binario mediante un debugger
- L'analisi della memoria ci ha consentito di capire la struttura del labirinto (**maze**)
- Siamo stati in grado di fornire la sequenza di input corretta per vincere la sfida



Vi siete divertiti?



Adesso tocca a voi



Seminari: 12 Maggio

- Nebula: Level 03
- Nebula: Level 05
- Nebula: Level 06
- Nebula: Level 08
- Nebula: Level 09
- Nebula: Level 11
- Nebula: Level 12
- Nebula: Level 14
- Nebula: Level 15



Seminari: 13 Maggio

- Nebula: Level 16
- Nebula: Level 17
- Nebula: Level 18
- Nebula: Level 19



Seminari: 19 Maggio

- DVWA: File Upload
- DWVA: Javascript
- Protostar: Stack Six
- Protostar: Stack Seven
- Protostar: Format One
- Protostar: Format Two
- Protostar: Format Three
- Protostar: Format Four
- CVE-2019-0708



Seminari: 20 Maggio

- Protostar: Heap Zero
- Protostar: Heap One
- Protostar: Heap Two
- Protostar: Heap Three



Seminari: 26 Maggio

- Protostar: Final Zero
- Protostar: Final One
- Protostar: Final Two
- CVE-2020-7247



Seminari: 27 Maggio

- Heist_ML_CTF_Challenge
- Morph su Cyberchallenge.IT
- Wordpress CVE su TryHackMe
- CVE-2023-4863



Seminari: 3 Giugno

- Fusion Level 03
- Fusion Level 04
- Fusion Level 05
- Fusion Level 06
- Fusion Level 07
- PARTY DI FINE CORSO

